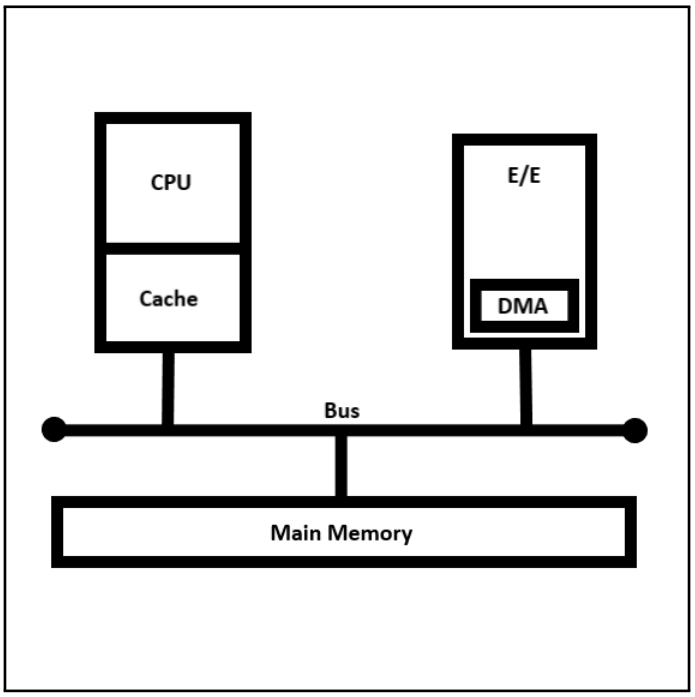
HY225 - Ασκήσεις 14 - csd5748

Άσκηση 14.1



α Όταν η συσκευή E/E στέλνει δεδομένα στη μνήμη μέσω DMA, ο επεξεργαστής διαβάζει τα σωστά αν γίνει cache miss (οπότε προσπελάζει την κύρια μνήμη) ή αν υπάρχει μηχανισμός που ενημερώνει την κρυφή μνήμη. Διαφορετικά, σε cache hit χωρίς ενημέρωση, διαβάζει παλιά και λανθασμένα δεδομένα ...

β Όχι, γιατί με πολιτική write-through κάθε εντολή store ενημερώνει αμέσως τη μνήμη. Άρα, όταν ξεκινά το DMA, η μνήμη έχει ήδη τα σωστά δεδομένα για τη συσκευή εξόδου ...

γ Με write-back, αν η έξοδος διαβάσει από τη μνήμη πριν τα δεδομένα περάσουν από την cache, ενδέχεται να είναι λανθασμένα. Αν έχουν ήδη γραφτεί τότε θα είναι σωστά ...

Άσκηση 14.3

[α] ii: 1 read: Η κρυφή μνήμη του Α έχει αστοχία οπότε ζητά την μεταβλητή από την κύρια μνήμη.
[α] iii: 1 read: Η κρυφή μνήμη του Β έχει αστοχία οπότε ζητά την μεταβλητή από την κύρια μνήμη.
[α] iv: Καμιά πράξη: Η κρυφή μνήμη Α ενημερώνεται με την τιμή 11 αλλά όχι η κύρια μνήμη (write-back).
[α] v: Καμιά πράξη: Η κρυφή μνήμη Β έχει ευστοχία (χρησιμοποιεί παλιά τιμή 0), οπότε δεν ζητά τίποτα.
[α] $*$: write-through: (iii) Κρυφή μνήμη Α ενημερώνει τιμή 11 παντού (iv) άρα την χρησιμοποιεί ο Β ...

[β] ii: 1 read: Κρυφή μνήμη Α αστοχία, ζητάει από κύρια μνήμη μέσω αρτηρίας και κατάσταση γίνεται S.
[β] iii: 1 read: Κρυφή μνήμη Β αστοχία, ζητά μεταβλητή από κρυφή μνήμη Α και κατάσταση γίνεται S.
[β] iv: 1 inv + 1 wr: Α γράφει 11 γίνεται (M), invalidate bus, κρυφή Β ακυρώνεται (I), κύρια ενημερώνεται.
[β] v: 1 read: Κρυφή του Β έχει αστοχία (I), οπότε ζητά από τον Α (M) και παίρνει 11*, κατάσταση Β (S).
[β] $*$: Με κατάσταση Ε: (ii) Α (E) αντί (S) αφού μόνος με αντίγραφο. (iii) Α (E) σε (S) λόγω Β ...

[γ] ii: 1 read: Κρυφή μνήμη Α αστοχία, ζητά μετ. από κύρια μνήμη, κατάσταση S (πρωτόκολο συνοχής MSI).
[γ] iv: 1 inv + 1 wr: Α γράφει 11, κατάσταση M, invalidate στο bus (χωρίς λόγο), κύρια μνήμη ενημερώνεται.
[γ] ii: 1 read: Κρυφή μνήμη Α αστοχία, ζητά μεταβλητή από κύρια μνήμη, κατάσταση Ε (μοναδικός)(MESI).
[γ] iv: 1 wr: Α γράφει 11, κατάσταση M, χωρίς invalidate (Β δεν έχει αντίγραφο), κύρια μνήμη ενημερώνεται.

Άσκηση 14.4

[α] Assembly risc-v with no instruction scheduling ...

```
* lw t0, 120          # Load a from DM[120] to t0.
* addi t1, t0, 2      # t1 = t0 + 2 (x = a + 2).
* sw t1, 128          # Store t1 (x) at DM[128].
* lw t2, 124          # Load b from DM[124] to t2.
* addi t3, t2, -1     # t3 = t2 - 1 (y = b - 1).
* sw t3, 132          # Store t3 (y) at DM[132].
```

[*] Pipeline (IF, ID, EX, MEM, WB): Load & Addi $\rightarrow$ 2 κύκλοι $\Rightarrow$ Χαμένοι κύκλοι $= 2 \times 2 = 4$ ...

[β] Assembly risc-v with instruction scheduling ...

```
* lw t0, 120          # Load a from DM[120] to t0.
* lw t2, 124          # Load b from DM[124] to t2.
* addi t1, t0, 2      # t1 = t0 + 2 (x = a + 2).
* addi t3, t2, -1     # t3 = t2 - 1 (y = b - 1).
* sw t1, 128          # Store t1 (x) at DM[128].
* sw t3, 132          # Store t3 (y) at DM[132].
```

[γ] Assembly risc-v with no pointers instruction scheduling ...

```
* lw t0, 0(x14)       # Load *pa from x14 to t0.
* addi t1, t0, 2      # t1 = t0 + 2 ((*px)=(*pa)+2).
* sw t1, 0(x16)       # Store t1 to x16 (*px).
* lw t2, 0(x15)       # Load *pb from x15 to t2.
* addi t3, t2, -1     # t3 = t2 - 1 ((*py)=(*pb)-1).
* sw t3, 0(x17)       # Store t3 to x17 (*py).
```

[*] Ο compiler δεν μπορεί να κάνει instruction scheduling αφού δεν γνωρίζει τις τιμές των pointers ...
[*] Εάν *px $\neq$ *pb τότε δεν υπάρχουν αλληλεξαρτήσεις $\Rightarrow$ μπορεί να υλοποιηθεί instruction scheduling ...
[*] Εάν *px $=$ *pb τότε υπάρχουν αλληλεξαρτήσεις $\Rightarrow$ απαγορεύεται υλοποίηση instruction scheduling ...

[δ] Η δεύτερη εντολή μπορεί να εξαρτάται από την πρώτη αν οι δείκτες δείχνουν στην ίδια διεύθυνση (1%).
[*] Στην αντίθετη περίπτωση που οι διευθύνσεις είναι διαφορετικές δεν υπάρχουν αλληλεξαρτήσεις (99%).
[*] Η επόμενη load μπορεί να εξαρτάται από την προηγούμενη store, υπό την προϋπόθεση *px $=$ *pb ...
[*] Έτσι, hardware πετυχαίνει δυναμική αναδιάταξη και καθυστερεί δεύτερη μέχρι ολοκληρωθεί πρώτη ...
[*] Τελικά, χρόνος χάνεται όταν βρισκόμαστε στο 1% των περιπτώσεων και όχι στο 99% ...

Άσκηση 14.5

α (Συνολικοί κύκλοι αναμονής) − (Χρήσιμες εντολές που εκτελούνται) = 80 − 8 = 72 κύκλοι.

β Το νήμα (πρόγραμμα) Α τρέχει σε ακριβώς τον ίδιο χρόνο, αλλά η συνολική απόδωση βελτιώνεται ...

γ Οι συνολικά χαμένοι κύκλοι είναι 0, ο επεξεργαστής δεν αδρανοποιείται, πάντα εκτελείται κάτι χρήσιμο.

ΤΕΛΟΣ
(ευχαριστώ)

LaTeX